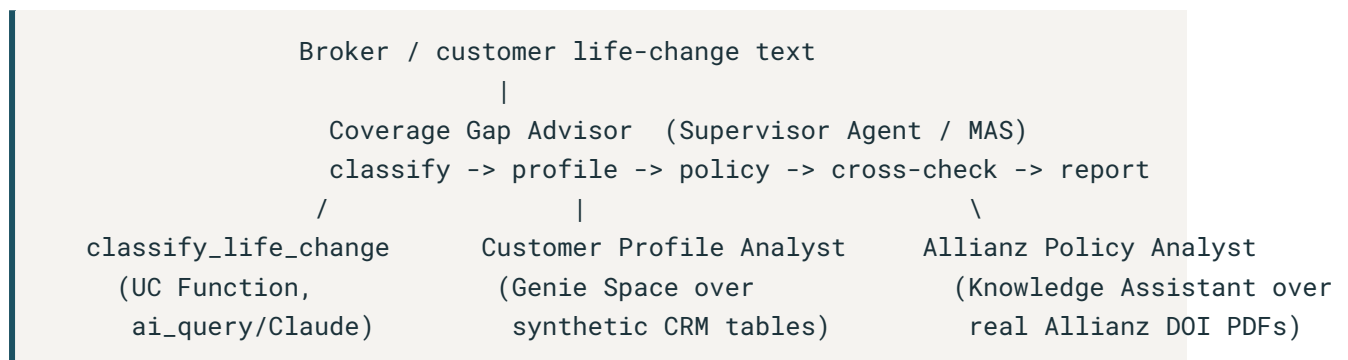


Allianz Coverage Gap Advisor – Build Guide

How the Coverage Gap Analysis demo was built on Databricks, end to end, so the team can recreate or extend it.

The demo is an **Agent Bricks Supervisor Agent** that reviews a customer's policy, their CRM profile, and a described life change, then produces a plain-language coverage gaps report with specific endorsement recommendations.

1. Architecture



Mapping to the customer brief:

- "Reads the policy PDF" -> Knowledge Assistant over the real Allianz Home + Motor Document of Insurance booklets, surfaced to the supervisor through a reliable `policy_lookup` UC function (an `ai_query` wrapper over the KA endpoint).
- "Loads the CRM profile" -> Genie Space over synthetic `customers / policies / assets / dependants / claims / life_events` tables.
- "Classifies the life change" -> `classify_life_change` UC function.
- "Cross-checks and drafts the report" -> Supervisor Agent orchestration.

All assets are in `mlops_pj.coverage_gap` on the `DEFAULT` (Azure) workspace.

2. Prerequisites

- Databricks CLI authenticated to the workspace (profile `DEFAULT`).
- A running SQL warehouse (we used `ec3b3b85811ecd2c`).
- Python with `databricks-sdk` . The Supervisor Agent API requires **`databricks-sdk` >= 0.103** (we used 0.114.0); Knowledge Assistant and Genie create APIs work from ~0.102.

- Foundation Model API access (endpoint `databricks-claude-sonnet-4-5` for the classifier).
- The real Allianz DOI PDFs (in `DOI/Home` and `DOI/Motor`).

Note: the Databricks dev-kit MCP server was not usable here because the workspace uses OAuth (`databricks-cli auth`) which the MCP server cannot use non-interactively. Everything below is done with the authenticated CLI / Python SDK instead.

3. Create the schema, volume, and upload the policy PDFs

```
databricks schemas create coverage_gap mlops_pj -p DEFAULT \
  --comment "Allianz Coverage Gap Analysis POV"
databricks volumes create mlops_pj coverage_gap policies MANAGED -p DEFAULT

databricks fs mkdir "dbfs:/Volumes/mlops_pj/coverage_gap/policies/home" -p DEFAULT
databricks fs mkdir "dbfs:/Volumes/mlops_pj/coverage_gap/policies/motor" -p DEFAULT

databricks fs cp "DOI/Home/0038733-2024 AZ Home DOI Final.pdf" \
  "dbfs:/Volumes/mlops_pj/coverage_gap/policies/home/Allianz_Home_DOI.pdf" -p DEFAULT
databricks fs cp "DOI/Motor/Allianz Silver (DOI).pdf" \
  "dbfs:/Volumes/mlops_pj/coverage_gap/policies/motor/Allianz_Motor_Silver_DOI.pdf" -p DE
databricks fs cp "DOI/Motor/Allianz Online Car Insurance DOI.pdf" \
  "dbfs:/Volumes/mlops_pj/coverage_gap/policies/motor/Allianz_Online_Car_DOI.pdf" -p DEFA
```

4. Create the synthetic CRM tables

The full DDL is in `sql/01_create_crm_data.sql` . Cover tiers and sublimits are aligned to the REAL Allianz DOI so the gaps are genuine:

- Home (Allianz Home DOI): Business equipment sublimit Bronze 1,000 / Silver 5,000 / Gold 10,000; Valuables single item 2,000 (all tiers); Liability to domestic employees Excluded on Bronze.
- Motor (Allianz Car DOI): default use class is Social, Domestic, Pleasure & Commuting; business / hire-and-reward use is excluded.

Run it with the helper (`sql/run_sql.py`):

```
python3 sql/run_sql.py sql/01_create_crm_data.sql
```

Tables created: `customers`, `policies`, `assets`, `dependants`, `claims`, `life_events`.

5. Create the `classify_life_change` UC function

`sql/02_create_function.sql` defines a function that calls a Foundation Model via `ai_query` to map a life change to insurance need categories:

```
CREATE OR REPLACE FUNCTION mlops_pj.coverage_gap.classify_life_change(life_change STRING)
RETURNS STRING
COMMENT 'Classifies a customer life change ... returns a categorised list with reasons.'
RETURN ai_query('databricks-claude-sonnet-4-5', CONCAT('You are an insurance needs classi
```

```
python3 sql/run_sql.py sql/02_create_function.sql
```

Test:

```
SELECT classify_life_change('I started a home catering business and bought a 4,800 pound
```

6. Create the Knowledge Assistant (Policy Analyst)

Script: `agents/create_ka.py`. It creates the assistant, adds a FILES knowledge source over the policy volume, and triggers indexing.

```
from databricks.sdk import WorkspaceClient
import databricks.sdk.service.knowledgeassistants as ka
w = WorkspaceClient(profile="DEFAULT")
a = w.knowledge_assistants.create_knowledge_assistant(
    knowledge_assistant=ka.KnowledgeAssistant(
        display_name="Allianz Policy Analyst", description="...", instructions="...")
w.knowledge_assistants.create_knowledge_source(parent=a.name,
    knowledge_source=ka.KnowledgeSource(display_name="Allianz DOI booklets",
    source_type="FILES", files=ka.FilesSpec(path="/Volumes/mlops_pj/coverage_gap/poli
```

```
python3 agents/create_ka.py
```

Gotcha: in SDK 0.102 `sync_knowledge_sources` sends no body and 400s. Trigger the sync directly instead:

```
databricks api post \
  "/api/2.1/knowledge-assistants/<KA_ID>/knowledge-sources:sync" -p DEFAULT --json '{}'
```

Indexing takes ~10–15 minutes. Wait for the knowledge source state to read `UPDATED` before demoing.

6b. Wrap the KA in a `policy_lookup` UC function

The supervisor calls the policy KA through a UC function rather than the native `knowledge_assistant` tool. In testing, the direct `knowledge_assistant` subagent returned empty content to the supervisor (the KA works perfectly when called standalone), whereas a UC function calling the KA endpoint via `ai_query` is reliable and fast (~15–25s) and returns clean, citable wording. Script/SQL: `sql/03_create_policy_lookup.sql`.

```
CREATE OR REPLACE FUNCTION mlops_pj.coverage_gap.policy_lookup(question STRING)
RETURNS STRING
COMMENT 'Looks up authoritative Allianz policy wording by querying the Policy Analyst KA
RETURN ai_query('ka-924e4b7c-endpoint', question);
```

Test it returns real DOI wording (section names and sublimits):

```
SELECT mlops_pj.coverage_gap.policy_lookup(
  'What is the business equipment sublimit on the Allianz Home policy and which section?'
```

7. Create the Genie Space (Customer Profile Analyst)

Script: `agents/create_genie.py`. It builds a `serialized_space` (version 2) listing the six tables, sample questions, and instructions, then calls `w.genie.create_space(...)`.

Key points:

- IDs inside `serialized_space` must be 32-char hex; collections are sorted by id.
- Pass the warehouse id and a title/description.

```
python3 agents/create_genie.py
```

8. Create the Supervisor Agent (Coverage Gap Advisor)

Script: `agents/create_mas.py`. Requires `databricks-sdk >= 0.103`; if your global SDK is older, install into an isolated folder and set `PYTHONPATH`:

```
pip install --target ../agentlibs "databricks-sdk==0.114.0"
PYTHONPATH="../agentlibs" python3 agents/create_mas.py
```

It creates the supervisor agent, then adds three tools and two examples:

```
import databricks.sdk.service.supervisoragents as sa
w.supervisor_agents.create_tool(parent=agent.name, tool_id="life_change_classifier",
    tool=sa.Tool(tool_type="uc_function",
        uc_function=sa.UcFunction(name="mlops_pj.coverage_gap.classify_life_change"), des
w.supervisor_agents.create_tool(parent=agent.name, tool_id="customer_profile_analyst",
    tool=sa.Tool(tool_type="genie_space", genie_space=sa.GenieSpace(id=GENIE_SPACE_ID), d
w.supervisor_agents.create_tool(parent=agent.name, tool_id="policy_lookup",
    tool=sa.Tool(tool_type="uc_function",
        uc_function=sa.UcFunction(name="mlops_pj.coverage_gap.policy_lookup"), descriptio
```

The supervisor `instructions` encode the workflow (classify → profile → policy lookup → cross-check → structured GBP report) and make the policy lookup **mandatory for every gap**, requiring the report to cite the policy section the lookup returns. See `agents/update_mas.py` and `agents/update_mas2.py` for the optimization pass (citation discipline, tighter output format, swap to `policy_lookup`).

Tip: `update_supervisor_agent / update_tool` take an `update_mask` as `sa.FieldMask(field_mask=["instructions"])` (a list of paths), not a plain string.

8b. RCA: why we use `policy_lookup` instead of the native Knowledge Assistant subagent

We tried hard to wire the KA as a native `knowledge_assistant` subagent (the documented GA pattern). It does not work in this workspace via the SDK. The root-cause investigation:

- The KA is healthy standalone: endpoint `ka-924e4b7c-endpoint` is `ACTIVE`, task type `agent/v1/responses`, backed by an `ONLINE` Vector Search endpoint. Direct queries (and `ai_query` against it) return correct, citable answers.
- Genie and UC-function subagents work through the supervisor; only the KA subagent fails.
- When the supervisor calls the native KA tool, the raw response contains the `function_call` for `policy_analyst` but **no `function_call_output` is ever returned for it** (the classifier and Genie return outputs normally). So the KA subagent call fails at the orchestration layer and is dropped – the model then retries and falls back to hedged wording.

What we ruled out:

- Tool wiring form: tested both `KnowledgeAssistant(knowledge_assistant_id=..., serving_endpoint_name=...)` and the documented id-only form. Same empty result.
- Permissions: the supervisor uses On-Behalf-Of auth (runs as the caller). The caller holds `CAN_MANAGE` on the KA endpoint, and we also granted `CAN_QUERY` to `users /`

`account users` . No effect.

– Stale provisioning: re-created the supervisor from scratch with the KA wired before first deploy, against the now-live KA. Still empty.

Conclusion: this is a platform-side orchestration issue for KA subagents. The documented remediation is the Agent Bricks UI **Update Agent** button (pulls server-side orchestration fixes), which is not exposed via the SDK. Until that is applied / the workspace is updated, the reliable approach is the `policy_lookup` UC function (section 6b): it calls the same KA endpoint via `ai_query` , runs under the SQL warehouse identity, and returns clean, citable wording every time. The standalone KA tile remains available for the "ask the policy directly" moment in the AI Playground.

To retry native KA later: open the Supervisor Agent in the Agent Bricks UI, click **Update Agent**, ensure the KA tile shows `CAN QUERY` , then re-run `agents/fix_native_ka.py` to swap `policy_lookup` back out for the native `policy_analyst` tool and re-test with `agents/test_advisor.py` .

9. Resource identifiers (this build)

Component	Identifier
Knowledge Assistant	KA id <code>924e4b7c-5165-44fb-8985-b6d6962f8a1d</code> , endpoint <code>ka-924e4b7c-endpoint</code>
Genie Space	<code>01f15eca8e531019982990a65ade688e</code>
UC Function (classifier)	<code>mlops_pj.coverage_gap.classify_life_change</code>
UC Function (policy lookup)	<code>mlops_pj.coverage_gap.policy_lookup</code> (<code>ai_query</code> over <code>ka-924e4b7c-endpoint</code>)
Supervisor Agent	MAS id <code>138f2af1-9ba2-4f8f-a93b-7d5aec8174a0</code> , endpoint <code>mas-138f2af1-endpoint</code>

10. Demo personas and prompts

Sarah Thompson (flagship – home catering business, Bronze Home)

Prompt to the advisor:

Sarah Thompson has started a home catering business and bought 4,800 pounds of equipment including a camera for food photography. Her niece helps at

weekend events. What are her coverage gaps and what should I recommend before her renewal?

Expected gaps:

1. Business equipment 4,800 vs Bronze sublimit 1,000 -> 3,800 GBP over-limit.
2. Business pursuits / business use exclusion -> no liability for food-related incidents.
3. Liability to domestic employees excluded on Bronze -> uncovered employer liability for the helper.

Recommended: home business / business equipment endorsement, business public/product liability, employers' liability.

Priya Patel (high-value item, Gold Home)

Priya Patel just got engaged and bought a 6,000 pound diamond ring. Any coverage gaps?

Expected: ring 6,000 vs valuables single item limit 2,000 -> 4,000 over; recommend specifying the item / valuables endorsement.

James O'Connor (motor business use, Allianz Silver)

James O'Connor has started doing food delivery for Deliveroo in his car. Any coverage gaps on his motor policy?

Expected: food delivery is business / hire-and-reward use, excluded under the current use class; recommend adding hire-and-reward / business use cover.

11. Where to demo

- AI Playground -> endpoint `mas-138f2af1-endpoint` for the full advisor.
- AI Playground -> `ka-924e4b7c-endpoint` for the policy assistant alone.
- Genie -> "Allianz Customer Profile Analyst" space for the CRM agent alone.
- Agents page shows the Knowledge Assistant and Supervisor Agent tiles.

12. Troubleshooting

- Dev-kit MCP returns null user / no-ops: it cannot use OAuth non-interactively. Use the CLI/SDK.

- Supervisor agent API missing in the SDK: upgrade to ≥ 0.103 (we used 0.114.0 via `--target ./agentlibs`).
- KA answers look empty: the knowledge source is still `UPDATING`. Wait for `UPDATED`.
- Native KA subagent returns nothing inside the supervisor (no `function_call_output`): known platform orchestration issue – use the `policy_lookup` UC function instead (see RCA in section 8b). Direct KA queries are unaffected.
- Genie can't resolve a customer: use the exact full name.
- Classifier errors: confirm `databricks-claude-sonnet-4-5` is available or change the model in `sql/02_create_function.sql`.